SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY



Design Document

RideLanka FleetHire

Web-Based Car Management Rental System

Group ID **2026-Y2-S1-KU-36**

MEMBER	STUDENT ID	CORE MODULE OWNERSHIP	SCRUM ROLE
S.M.H.C.B.D.rambukpitiya	IT25103736	User & Customer Management	Product Owner
I.K.S.P.Jayawardhana	IT25103732	Vehicle Management	Scrum Master
W.M.K.K.G.N.A.Weerasingha	IT25103737	Booking & Reservation Management	Developer
R.M.M.D.Bhagyamathi	IT25103733	Payment Management	Developer
Rajapaksha R.M.R.D	IT25103734	Feedback & Support Management	Developer
K.A.K.Dhananjaya	IT25103735	Fleet Management	Developer

Submitted 19 September 2026

Software Engineering | SE2030

B.Sc. (Hons) in Information Technology

Table of Contents

Table of Contents 2
1. Agile Sprint Summaries 3
Sprint delivery snapshot 3
Delivered and carried work 3
Quality gates and verification evidence 3
Planned Phase 3 scope 3
Backlog item index 4
2. System-Wide Use Case Architecture 5
Boundary and scope 5
Unified actor taxonomy 5
Relationship mechanics and structural annotations 5
3. Class Diagram & Bounded Context Breakdown 6
Structural notation and association rules 6
Domain grouping by bounded context 6
Context 1 — User & Customer Management 6
Context 2 — Vehicle Management & Fleet Scheduling 6
Context 3 — Booking & Reservation Management 6
Context 4 — Payment Management & Financial 7
Context 5 — Feedback, Support & Messaging 7
Core domain patterns and enterprise mechanisms 7
4. Activity Diagram Flow Breakdown Matrices 7
VM03 — Mark Vehicle Out of Service 7
BR04 — Reserve Vehicle & Confirm Booking 8
SM06 — Publish Approved Rental Schedule 8
PM03 — Verify Payment Status Before Voucher Issue 9
NM01 — Configure & Activate Notification Template 9
UM03 — Create Staff Account with Predefined Role 9
5. Ethical Considerations & Security Architecture 10
5.1 Privacy & cryptographic data minimization 10
5.2 Security as an obligation & defense-in-depth 10
5.3 Fairness & algorithmic concurrency 11
5.4 Comprehensive accessibility & universal UX design 11
5.5 Role-based access control (RBAC) governance matrix 11
5.6 Academic integrity & development transparency 11

1. Agile Sprint Summaries

Twenty-four product backlog items were drawn from six personas, ranked by business priority and dependency constraints, and executed across four two-week iterations. Completion status was evaluated directly against the source codebase and the automated integration build gate rather than against a manual task board.

Sprint delivery snapshot

- Total planned PBIs: 24 items
- Accepted / delivered: 22 items (91.6% completion rate)
- Carried to Phase 3: 2 items (8.4% scope deferred)
- Automated test pass rate: 100% (85 backend tests, 13 frontend E2E / accessibility tests)

Sprint	Sprint Goal	Scheduled PBIs	Completed	Effort (pts)	Status
Sprint 1	Controlled access & valid fleet search-to-reserve	21, 10, 09, 01, 02, 03	6 / 6	18	Closed Complete
Sprint 2	Secure payment path with auditable dispatch info	22, 14, 13, 04, 20, 17	6 / 6	21	Closed Complete
Sprint 3	Maintenance comms, support, and accountable refunds	23, 11, 18, 05, 06, 15	6 / 6	19	Closed Complete
Sprint 4	System recovery, reconciliation, and reporting	12, 07, 08, 16, 19, 24	4 / 6	22	Partial / Deferred

Delivered and carried work

Twenty-two items met all acceptance criteria. Sprints 1 through 3 were delivered in full. Two Sprint 4 items were carried into Phase 3 as pending work:

- PBI-12 (Nominate Replacement Vehicle): replacement nomination relies on automated waitlist offer expiration and real-time state conversion, which is still undergoing engine tuning.
- PBI-24 (User-Access Report for Selected Role): the filtered UI views are complete; multi-format export pipelines (CSV, XLSX, PDF) with cryptographic sign-off are still in development.

Technical limitation note: PBI-20 (QR validation) accepts physical hardware scanners, pasted payloads, or manual reference IDs; embedded device camera capture remains in progress.

Quality gates and verification evidence

- Backend pipeline: 85 Spring Boot unit/integration tests executed with 0 failures. Static analysis enforced via Checkstyle, SpotBugs, ArchUnit structure verification, and a full application context initialisation.
- Frontend pipeline: strict TypeScript compilation with zero warnings, zero ESLint defects, 5 WCAG accessibility unit tests, and 8 cross-browser Playwright E2E suites.
- Database verification note: SQL Server migration tests were bypassed on the host environment due to unmounted Docker engines; this is recorded as an explicit audit gap rather than assumed to pass.

Planned Phase 3 scope

- Live SQL Server concurrent lock resolution for overlapping booking attempts.
- Implementation of schedule lease acquire/release mechanisms and lifecycle state transitions.
- Camera-based client-side QR stream decoding.
- Integration with production SMS (Text.lk) and Email (SMTP) notification channels.

Backlog item index

Every backlog item is listed below so the sprint table can be read on its own. “Sprint” records when the item was scheduled; “Done” means accepted against the current build.

PBI	Backlog Item	Sprint	Target Module	Acceptance Status
PBI-01	Search available vehicles	Sprint 1	Booking Management	Accepted (Done)
PBI-02	View fleet inventory & rates	Sprint 1	Vehicle Management	Accepted (Done)
PBI-03	Email and SMS booking alerts	Sprint 1	Feedback & Support	Accepted (Done)
PBI-04	Retrieve the rental QR voucher	Sprint 2	Booking Management	Accepted (Done)
PBI-05	Locate a booking by identifier	Sprint 3	Booking Management	Accepted (Done)
PBI-06	Record a customer complaint	Sprint 3	Feedback & Support	Accepted (Done)
PBI-07	Place a customer on waiting list	Sprint 4	Booking Management	Accepted (Done)
PBI-08	Re-send an approved message	Sprint 4	Feedback & Support	Accepted (Done)
PBI-09	Publish vehicle rental schedule	Sprint 1	Fleet Management	Accepted (Done)
PBI-10	Conflict warning before publishing	Sprint 1	Fleet Management	Accepted (Done)
PBI-11	Record a revised return time	Sprint 3	Vehicle Management	Accepted (Done)
PBI-12	Nominate a replacement vehicle	Sprint 4	Fleet Management	Carried (Pending)
PBI-13	Verify payment before booking issue	Sprint 2	Payment Management	Accepted (Done)
PBI-14	Flag a duplicate payment attempt	Sprint 2	Payment Management	Accepted (Done)
PBI-15	Record a refund decision	Sprint 3	Payment Management	Accepted (Done)
PBI-16	Reconcile by transaction ID	Sprint 4	Payment Management	Accepted (Done)
PBI-17	View vehicle operational details	Sprint 2	Vehicle Management	Accepted (Done)
PBI-18	Publish maintenance disruption notice	Sprint 3	Vehicle Management	Accepted (Done)
PBI-19	Mark an affected vehicle unavailable	Sprint 4	Vehicle Management	Accepted (Done)
PBI-20	Validate QR voucher at handover	Sprint 2	Booking Management	Accepted (Done) *
PBI-21	Create a staff account with a role	Sprint 1	User Management	Accepted (Done)
PBI-22	Inspect system audit trail	Sprint 2	User Management	Accepted (Done)
PBI-23	Configure a notification template	Sprint 3	Feedback & Support	Accepted (Done)

PBI	Backlog Item	Sprint	Target Module	Acceptance Status
PBI-24	User-access report for a role	Sprint 4	User Management	Carried (Pending)

* PBI-20 is accepted with the limitation stated above: scanner, pasted payload or manual reference, but not yet camera decoding.

2. System-Wide Use Case Architecture

Rather than fragmenting the system model across isolated per-module views, RideLanka FleetHire uses a single unified use case model. As a modular monolith running over one relational store, cross-boundary interactions — a booking that reaches payment, a payment that issues a voucher, a schedule change that reaches notification — need to be visible on a single diagram rather than hidden by a module split.

Boundary and scope

- System boundary: RideLanka FleetHire Core Platform
- Total use cases: 38 functional cases
- Internal staff actors: 9 specialised roles inheriting from an abstract Staff User
- External infrastructure actors: PayHere Payment Gateway, Notification Delivery Service (Text.lk / SMTP)
- Relationships enforced: 6 «include», 3 «extend»

Unified actor taxonomy

All staff actors derive from an Abstract Staff User, which owns the universal access use cases every staff member needs — UC-UM01: Authenticate User and UC-UM02: Recover Password.

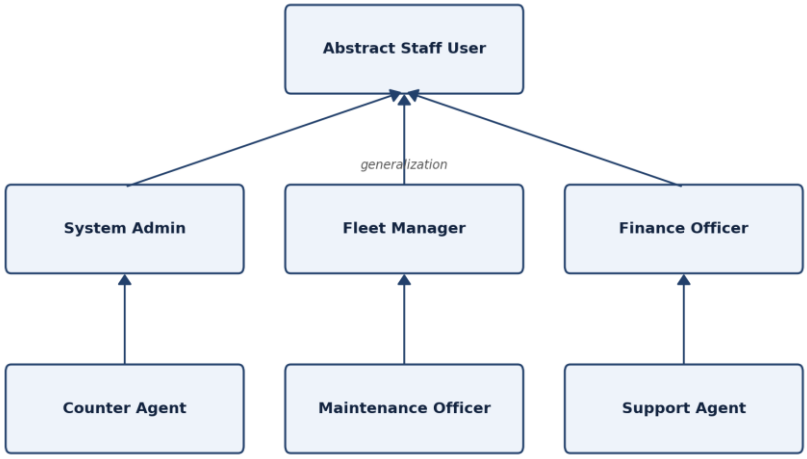


Figure 1 Unified actor generalization tree

Relationship mechanics and structural annotations

- «include» — applied where functionality executes every time the parent case executes, e.g. UC-BR04: Confirm Booking includes UC-PM03: Verify Payment Status.
- «extend» — applied where execution is conditional:
- Extension point OnCapacityExceeded inside UC-BR01: Search Vehicles, extending to UC-BR07: Join Waiting List.

- Extension point OnDisruptionDetected inside UC-VM03: Out of Service, extending to UC-FM12: Nominate Replacement.
- Extension point OnPaymentFailure inside UC-PM01: Process Payment, extending to UC-PM04: Flag Duplicate.

Full use case diagram: insert the team's complete 38-use-case / 14-actor diagram here (recommended tools: draw.io, Lucidchart, or Visio), using the actor tree and relationship rules above as its structural backbone.

3. Class Diagram & Bounded Context Breakdown

The class diagram reflects the implemented Java domain layer. It comprises 33 domain entities and 6 state-enforcing enumerations.

Structural notation and association rules

All domain classes use the standard UML three-compartment representation. Attributes state access visibility (+, #, –) and explicit data types; method signatures list input parameters. Composition (filled diamond) is reserved for lifecycle-dependent child entities, while aggregation (open diamond) represents shared associations. Encrypted attributes are appended with “Ciphertext”.

Domain grouping by bounded context

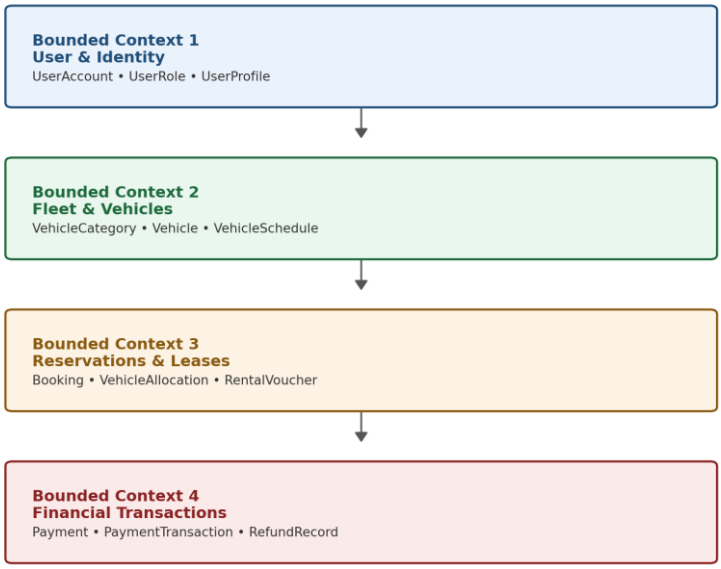


Figure 2 Bounded-context relationship overview

Context 1 — User & Customer Management

Handles principal authentication, role-based privilege mappings, and customer identity profiles. Sensitive identity fields (nicCiphertext, drivingLicenseCiphertext) are stored encrypted at rest using AES-256-GCM.

Context 2 — Vehicle Management & Fleet Scheduling

Models physical vehicle assets, maintenance logs, operational availability, and publication schedules. Vehicle entities track operational states: AVAILABLE, IN_SERVICE, MAINTENANCE, DECOMMISSIONED.

Context 3 — Booking & Reservation Management

Coordinates temporal vehicle locks, reservation life cycles, waiting-list queues, and encrypted rental QR voucher generation.

Context 4 — Payment Management & Financial

Processes transaction verification, idempotency checking, gateway callback reconciliation, and structured refund decisions.

Context 5 — Feedback, Support & Messaging

Manages customer complaint tracking, notification templates, and outbox notification delivery pipelines.

Core domain patterns and enterprise mechanisms

Pattern	Where and why it is used
Repository / DAO	Abstracts SQL interaction; interfaces (e.g. BookingRepository) are decoupled from execution mechanics (JdbcBookingDao), keeping domain rules independent of the database layer.
Transactional Outbox	Guarantees event-emission atomicity: OutboxEvent records commit alongside business entities in a single DB transaction, before OutboxProcessor dispatch.
Strategy & Adapter	Decouples external services via uniform interfaces (SmsProvider, EmailProvider), allowing live production services (Text.lk) and test mocks to swap seamlessly.
Full class diagram: insert the team's 33-entity / 6-enumeration diagram here, grouped by the four bounded contexts illustrated above plus the Feedback & Support entities.	

4. Activity Diagram Flow Breakdown Matrices

One control-flow breakdown is provided per major function. Each member modelled the use case in their own module carrying the most decision logic — the flows where a wrong branch would corrupt data, double-charge a customer, or expose something it should not.

Figure	Target Process Workflow	Module Ownership	Primary Author
Fig 3.0	VM03: Mark Vehicle Out of Service	Vehicle Management	[Member 2 Name]
Fig 4.0	BR04: Reserve & Confirm Booking	Booking Management	[Member 3 Name]
Fig 5.0	SM06: Publish Rental Schedule	Fleet Management	[Member 6 Name]
Fig 6.0	PM03: Verify Payment Status	Payment Management	[Member 4 Name]
Fig 7.0	NM01: Configure Message Template	Support Management	[Member 5 Name]
Fig 8.0	UM03: Provision Staff Account	User Management	[Member 1 Name]

VM03 — Mark Vehicle Out of Service

Module ownership: Vehicle Management | Primary author: [Member 2 Name]

Step	Actor / Swimlane	Executed Action / Decision	Pre-Conditions / Constraints	Next
VM03-1	Fleet Operator	Submits out-of-service request for vehicle.	Operator authenticated (OPERATIONS).	VM03-2
VM03-2	System Service	Verifies target vehicle current status in DB.	Vehicle must be in AVAILABLE state.	VM03-3
VM03-3	System Decision	[Guard]: Active reservations in window?	YES → VM03-4a / NO → VM03-4b	4a/4b

Step	Actor / Swimlane	Executed Action / Decision	Pre-Conditions / Constraints	Next
VM03-4a	System Service	Flag schedule conflict; trigger alert event.	Outbox event generated for scheduler.	VM03-5
VM03-4b	System Service	Update status to OUT_OF_SERVICE.	DB row lock acquired.	VM03-5
VM03-5	System Service	Merge node: consolidate path; write AuditLog.	Append-only security audit log entry.	End

BR04 — Reserve Vehicle & Confirm Booking

Module ownership: Booking Management | Primary author: [Member 3 Name]

Step	Actor / Swimlane	Executed Action / Decision	Pre-Conditions / Constraints	Next
BR04-1	Customer	Initiates vehicle checkout with dates/type.	User authenticated; valid payload.	BR04-2
BR04-2	System Service	Attempts pessimistic row-lock on schedule.	Transaction timeout set to 3000ms.	BR04-3
BR04-3	System Decision	[Guard]: Lock acquired successfully?	YES → BR04-4 / NO → BR04-7	4/7
BR04-4	System / PayHere	Processes transaction via PayHere Gateway API.	Valid payment token received.	BR04-5
BR04-5	System Service	Fork node: split into concurrent tasks.	Transaction committed.	6a/6b
BR04-6a	System Service	Task A: construct hash & render QR voucher.	SHA-256 HMAC signature calculation.	BR04-6c
BR04-6b	System Service	Task B: write payload to Outbox queue.	Persists Outbox entity in DB.	BR04-6c
BR04-6c	System Service	Join node: recombine asynchronous tasks.	Both task paths complete execution.	BR04-7
BR04-7	System Service	Merge node: consolidate path; return JSON.	Output structured response payload.	End

SM06 — Publish Approved Rental Schedule

Module ownership: Fleet Management | Primary author: [Member 6 Name]

Step	Actor / Swimlane	Executed Action / Decision	Pre-Conditions / Constraints	Next
SM06-1	Fleet Scheduler	Drafts new schedule publication for fleet.	Scheduler authenticated (SCHEDULER).	SM06-2
SM06-2	System Service	Executes timeline overlap algorithm on DB.	Temporal range evaluation.	SM06-3
SM06-3	System Decision	[Guard]: Timeline overlap detected?	NO → SM06-4 / HAS → SM06-7	4/7
SM06-4	Operations Mgr	Reviews pending schedule draft details.	User authenticated (OPERATIONS).	SM06-5
SM06-5	Ops Decision	[Guard]: Manager approval granted?	APPROVED → 6a / REJECTED → 6b	6a/6b
SM06-6a	System Service	Transitions status to PUBLISHED.	Atomic state transition.	SM06-7
SM06-6b	System Service	Records rejection reason code; notifies user.	State set to REJECTED.	SM06-7
SM06-7	System Service	Merge node: recombine branches; return response.	Operational audit log recorded.	End

PM03 — Verify Payment Status Before Voucher Issue

Module ownership: Payment Management | Primary author: [Member 4 Name]

Step	Actor / Swimlane	Executed Action / Decision	Pre-Conditions / Constraints	Next
PM03-1	PayHere Gateway	Transmits payment notification callback.	External HTTPS POST request.	PM03-2
PM03-2	System Service	Computes SHA-256 signature verification.	Uses configured merchant secret key.	PM03-3
PM03-3	System Decision	[Guard]: Digital signature payload valid?	VALID → 4 / INVALID → 7	4/7
PM03-4	System Service	Checks PaymentTransaction for gatewayTxnId.	Idempotency lock validation.	PM03-5
PM03-5	System Decision	[Guard]: Transaction ID unique?	UNIQUE → 6 / DUPLICATE → 7	6/7
PM03-6	System Service	Fork node: parallelize DB persistence steps.	Splitting execution paths.	6a/6b
PM03-6a	System Service	Path A: persists Payment as VERIFIED.	Writes primary payment record.	PM03-6c
PM03-6b	System Service	Path B: persists callback trace in log.	Writes raw payload trace.	PM03-6c
PM03-6c	System Service	Join node: synchronizes parallel writing.	Synchronizes DB state.	PM03-7
PM03-7	System Service	Merge node: unifies control paths; issues ACK.	Returns HTTP 200 OK or 400 Bad Request.	End

NM01 — Configure & Activate Notification Template

Module ownership: Feedback & Support Management | Primary author: [Member 5 Name]

Step	Actor / Swimlane	Executed Action / Decision	Pre-Conditions / Constraints	Next
NM01-1	Support Agent	Edits notification template & parameters.	Authenticated (SUPPORT).	NM01-2
NM01-2	System Service	Evaluates string syntax against schema.	Regex structure check.	NM01-3
NM01-3	System Decision	[Guard]: Template markup syntax valid?	VALID → 4 / INVALID → 6	4/6
NM01-4	System Service	Validates placeholders against schema keys.	Checks required parameter keys.	NM01-5
NM01-5	System Decision	[Guard]: Placeholders match domain schema?	MATCHED → 5a / UNMATCHED → 6	5a/6
NM01-5a	System Service	Sets version ACTIVE; archives prior version.	Atomic DB update.	NM01-6
NM01-6	System Service	Merge node: recombines paths; returns UI status.	Visual UI state update.	End

UM03 — Create Staff Account with Predefined Role

Module ownership: User & Customer Management | Primary author: [Member 1 Name]

Step	Actor / Swimlane	Executed Action / Decision	Pre-Conditions / Constraints	Next
UM03-1	System Admin	Submits staff creation form with role.	Admin authenticated (ADMIN).	UM03-2
UM03-2	System Service	Queries database for existing user email.	Exact match query.	UM03-3
UM03-3	System Decision	[Guard]: Email address unique in database?	UNIQUE → 4 / EXISTS → 6	4/6
UM03-4	System Service	Hashes temp password using Argon2id.	Parameters: m=65536, t=3, p=4.	UM03-5
UM03-5	System Service	Persists UserAccount, UserRole, writes audit.	Direct database transaction.	UM03-6
UM03-6	System Service	Merge node: responds with unified status.	Anti-enumeration standard response.	End

Optional: each matrix above can additionally be rendered as a swimlane activity diagram (as in the reference example) using draw.io or Visio, with one lane per actor/swimlane column.

5. Ethical Considerations & Security Architecture

5.1 Privacy & cryptographic data minimization

- Field-level encryption: NIC, passport, driving licence details, and refund account numbers are encrypted at rest using AES-256-GCM. Encryption keys are injected exclusively through secure environment variables at runtime, so no plaintext sensitive attribute reaches database backups or repository sources. Unique nonces bind encrypted values to specific entity IDs.
- One-way token hashing: operational verification tokens, recovery keys, reset identifiers, and rental QR payloads are stored using SHA-256 hashing.
- Zero cardholder data storage: credit/debit card numbers, CVV codes, and banking credentials are handled entirely within PayHere's PCI-DSS-compliant hosted pages; RideLanka FleetHire processes payment tokens exclusively.
- Data scoping: access permissions limit viewable data based on operational necessity — support representatives see masked identity fields.

5.2 Security as an obligation & defense-in-depth

- Authentication standards: passwords require a minimum of 12 characters, are checked against a breach-list denylist, and are hashed with Argon2id (m=65536 KB, t=3, p=4). Five consecutive failures trigger a 15-minute lock on both the account and the originating IP address.
- Multi-layer enforcement: security boundaries are checked independently across API controllers, domain services, database queries, and view components.
- Database isolation via three separate accounts — migrator (DDL/schema updates), runtime (restricted DML), and reporting (read-only).
- Audit logging: system modification actions generate append-only audit trail records capturing actor ID, timestamp, target entity, and IP address.

5.3 Fairness & algorithmic concurrency

- Fair booking lock engines: concurrent vehicle booking requests are handled using database row locks and unique constraints to eliminate race conditions.
- Deterministic pricing: rental rates are computed server-side from immutable rate snapshots; client-supplied rate calculations are rejected.
- Segregation of duties: administrative accounts (ADMIN) cannot modify their own assigned roles; maintenance actions require operational review (OPERATIONS), and refund transactions require financial approval (FINANCE).

5.4 Comprehensive accessibility & universal UX design

- Responsive layouts maintain functional integrity from 320px to 1440px viewports and withstand zoom levels up to 400% without loss of content.
- Semantic landmarks, visible focus rings, focus-trap controls in modal dialogs, and ARIA live regions (aria-live="polite") support assistive navigation.
- Fleet availability offers an accessible tabular list view alongside the visual grid picker, ensuring full keyboard-navigable access.
- State transitions, errors, and availability indicators use textual labels and icon overlays alongside colour, supporting colour-blind users.

5.5 Role-based access control (RBAC) governance matrix

System Function	ADMIN	OPS	SCHED.	FINANCE	COUNTER	CUSTOMER
Provision User Account	X					
Inspect Audit Logs	X					
Mark Out-Of-Service		X				
Publish Fleet Schedule		X	X			
Issue Refund Decision				X		
Process Handover QR					X	
Create Reservation					X	X

5.6 Academic integrity & development transparency

Generative AI tools were used during development strictly within permitted module guidelines: assisting in domain research, preliminary code prototyping, documentation formatting, and debugging support. Core system logic, architectural implementation, state machines, and concurrency controls were developed, integrated, and verified directly by team members. Implementation details and limitations reported reflect the operational codebase baseline as of 19 September 2026.